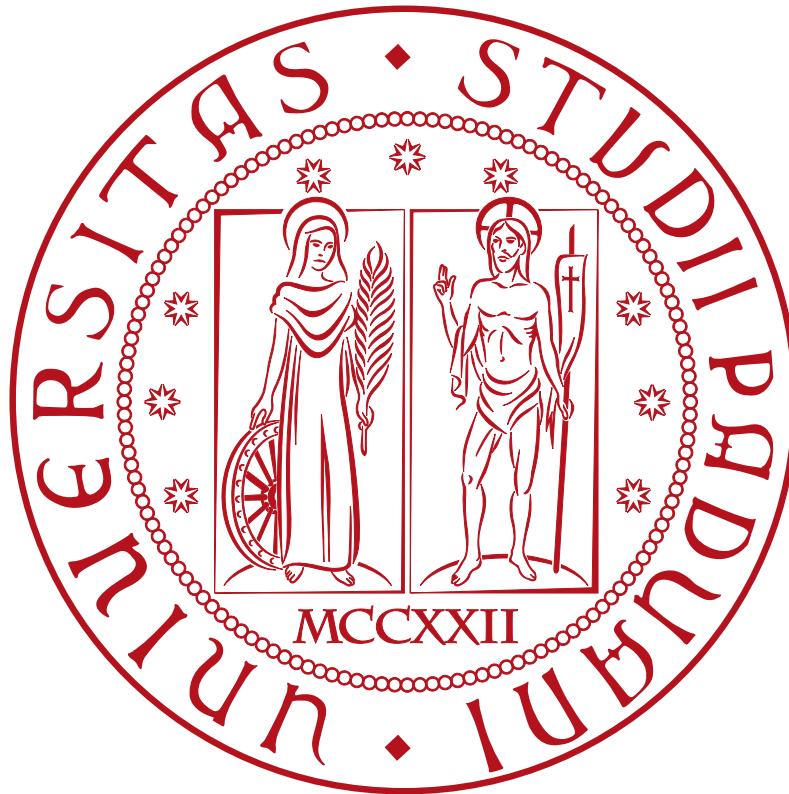


University of Padua



AIoT Basics using Containerization

Exercises 1, 2, 3, 4, 5

Student: Matteo Squarzoni

Student ID: 2197481

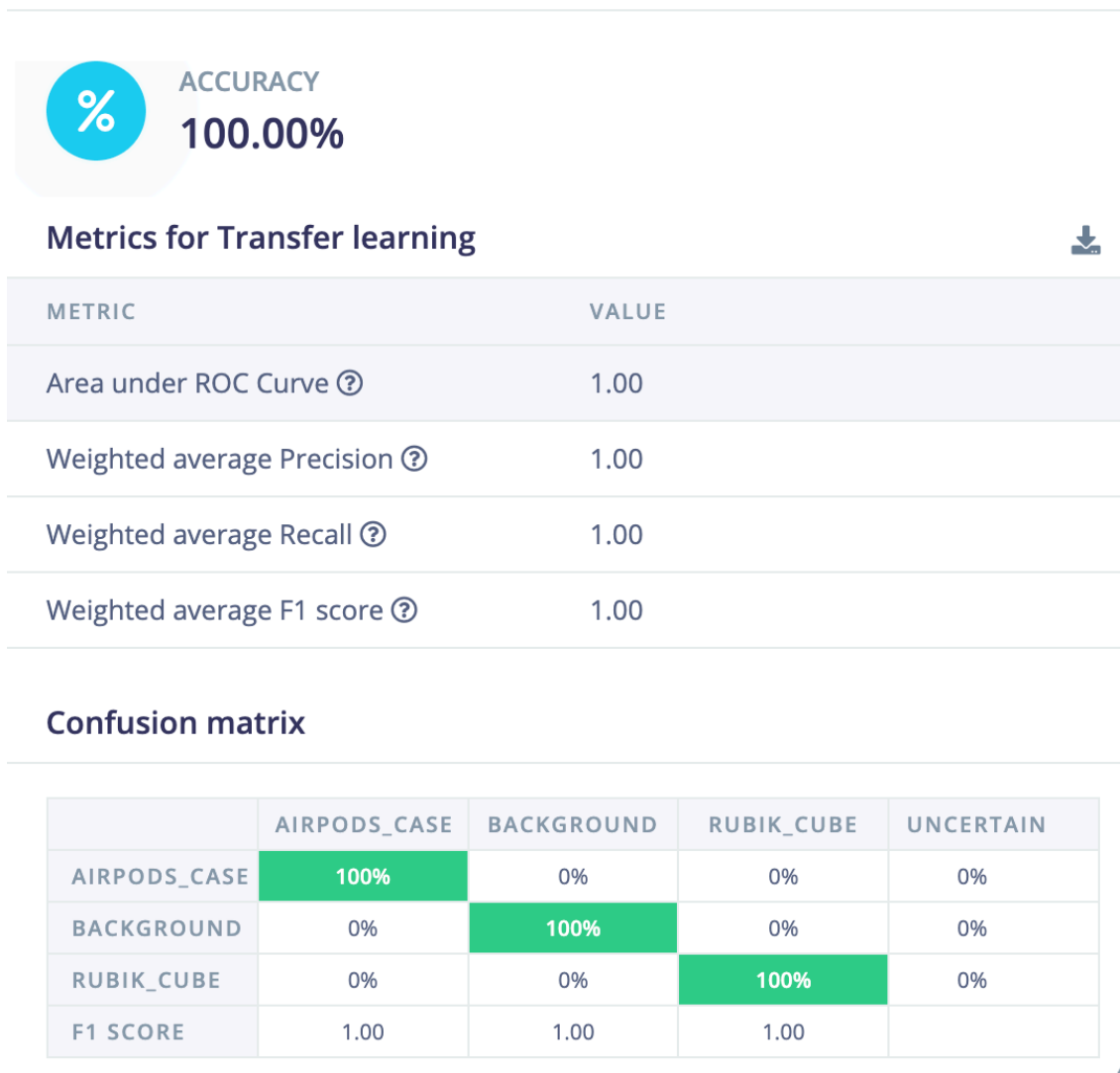
Contents

1. Exercise 1	1
2. Exercise 2	3
3. Exercise 3	4
4. Exercise 4 and 5	5

1. Exercise 1

To make the first exercise I decided to classify 2 different items: a Rubik's cube and the AirPods' case. I took 40 pictures for each item, and also I took 40 pictures of the background.

After training the model with an 80/20 split, I got the following results in the model testing section of Edge Impulse:



Note that the accuracy is very high, but the dataset is very small and the task is very easy.

After the deployment here is the QR code to test the model on your phone:



Figure 1: QR code of the model

2. Exercise 2

For the second exercise, I built a dashboard to monitor some metrics of the CPU and memory usage of my device. In particular I used the plugins of Telegraf to collect the CPU and memory usage and I sent the data to InfluxDB. Then I created a dashboard in Grafana to visualize the data. All the components are running in Docker containers.

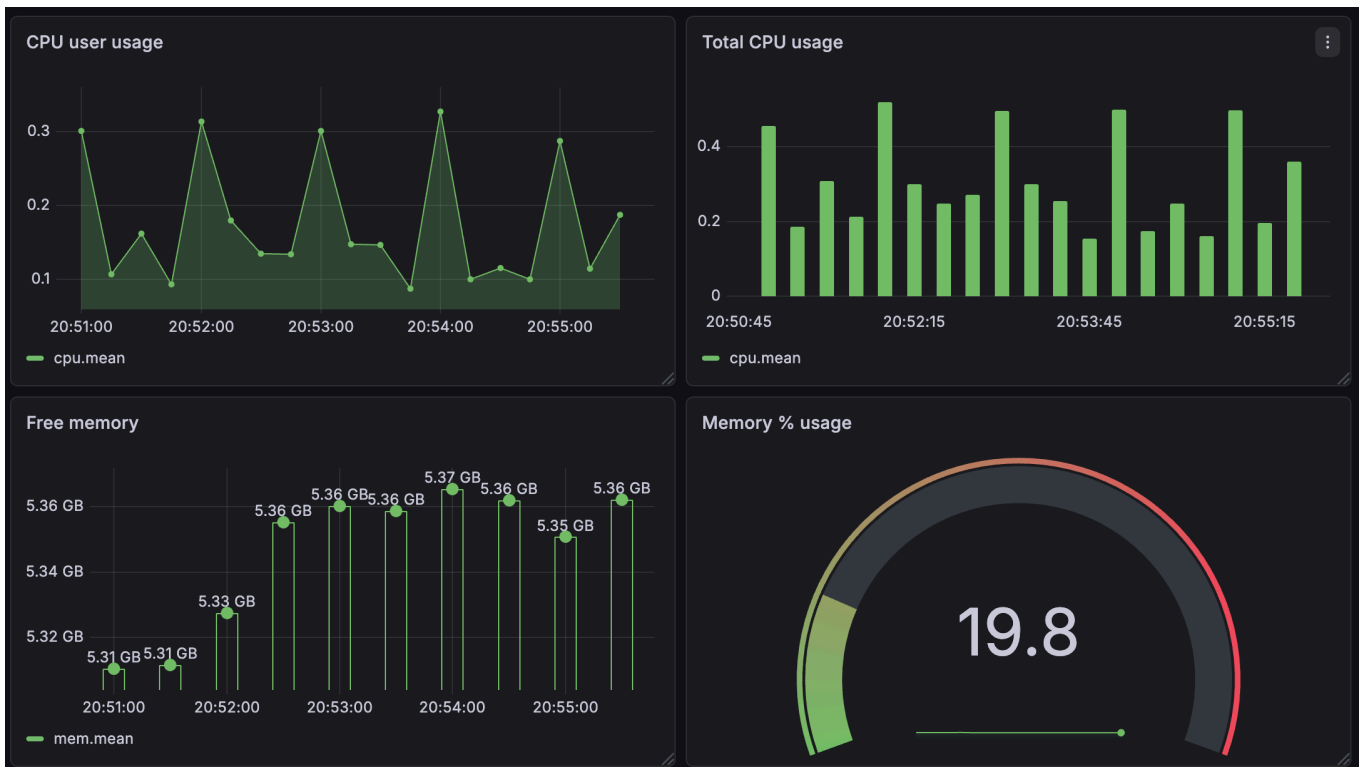


Figure 2: Grafana dashboard to monitor CPU and memory usage

The dashboard is very simple:

- on the upper left corner there is a graph that shows the CPU user usage over time;
- on the upper right corner there is a graph that shows the total CPU usage over time;
- on the lower left corner there is a graph that shows the amount of free memory over time;
- on the lower right corner there is a graph that shows the amount of used memory in percentage.

3. Exercise 3

In the third exercise I wrote a docker-compose file to run two MQTT brokers simultaneously in two different containers.

```
1  services:
2    broker_a:
3      image: eclipse-mosquitto:latest
4      container_name: broker_a
5      ports:
6        - "1883:1883"
7      volumes:
8        - ./mosquitto:/mosquitto
9      networks:
10       - pubsub-net
11
12    broker_b:
13      image: eclipse-mosquitto:latest
14      container_name: broker_b
15      ports:
16        - "1884:1883"
17      volumes:
18        - ./mosquitto:/mosquitto
19      networks:
20       - pubsub-net
21
22  networks:
23    pubsub-net:
24      driver: bridge
```

Listing 1: docker-compose file

After running the docker-compose file the two brokers were running simultaneously, and I was able to use the efrecon/mqtt-client image to publish and subscribe to each broker separately by specifying the appropriate port (1883 for broker_a and 1884 for broker_b).

Here is the link to the GitHub repository with the code of the docker-compose file: https://github.com/MatteoSquarz/AIoT-assignments/blob/main/mqtt_brokers/docker-compose.yml

4. Exercise 4 and 5

For the last two exercises, I wrote a Telegram bot that can be used to get the current temperature in Padua. The bot uses the Open-Meteo API to get the current weather data and it sends the temperature to the user when they send the “/getdata” command. Furthermore, the bot can be used to get some other basic statistics about the weather in Padua, such as the average, the minimum and maximum temperature, and a history of the last 10 measurements.

In the implementation of the bot, I used a REST-based API to get the current weather data from the Open-Meteo server, and I used a local SQLite database to store the temperature measurements.

The difference between a REST-based API (pull model) and a MQTT-based API (push model) is that the REST-based API uses a synchronous Request/Response architecture over HTTP, while the MQTT-based API uses an asynchronous Publish/Subscribe architecture.

The code of the bot is available at this GitHub repository: https://github.com/MatteoSquarz/AIoT-assignments/tree/main/telegram_bot

In the following Figure 3 you can see the Telegram bot in action, showing the current temperature in Padua and some statistics about it.

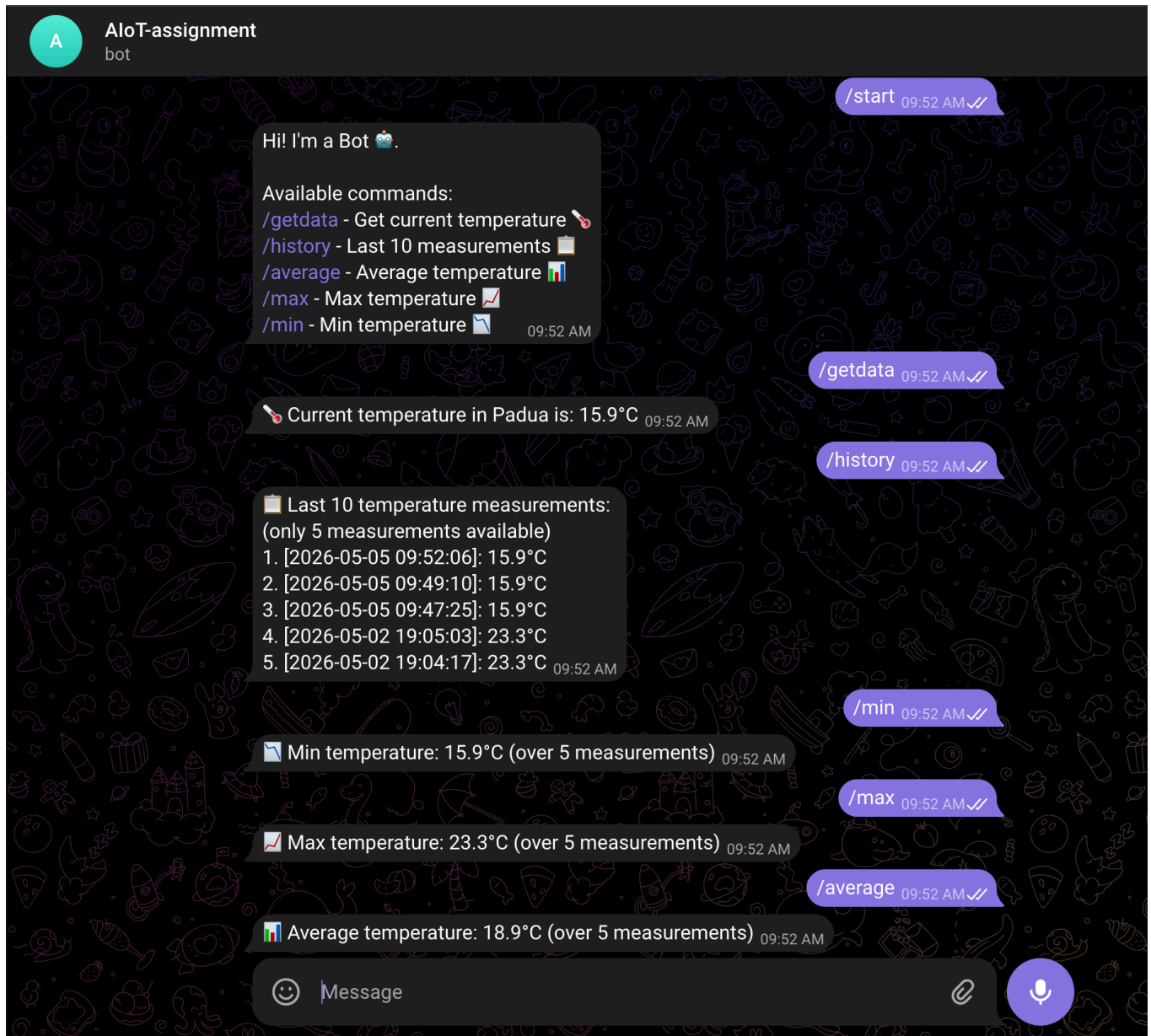


Figure 3: Telegram bot to get the current temperature in Padua and some statistics about it.